

SYS-304 Milestone 3: Architectural Scaling & Optimization

House Price Prediction service (Ames, Iowa · XGBoost) · Phase 2 (naive) vs Phase 3 (optimised) · Nunthatinn · September 2026

Result. The model is **917× faster** after ONNX export and distillation, with a validation RMSE change of +0.0008 (log price). At 64 concurrent clients the service handles **41×** the throughput of Phase 2 on unique traffic and **87×** on repeated traffic. p95 latency drops from 1,369 ms to 43 ms. Every load test finished with zero errors.

At 64 concurrent clients	Phase 2 (naive)	Phase 3 (optimised)	Change
Throughput, cold traffic (all unique)	65 req/s	2,680 req/s	41×
Throughput, warm traffic (repeated houses)	68 req/s	5,947 req/s	87×
p95 latency, cold	1,369 ms	43.0 ms	32× lower
p95 latency, warm	1,273 ms	18.0 ms	71× lower
Single-client p50 latency (end-to-end HTTP)	12.5 ms	1.34 ms	9× lower
Model inference p50 (in-process)	10,456 µs	11 µs	917× lower
Memory per serving process	216 MB (1 process)	130 MB (1 worker)	40% less
Validation RMSE (log price, serving protocol)	0.2127	0.2135	+0.0008
Errors during all load tests	0	0	—

1 · Setup and methodology

- **Hardware:** 2 vCPU x86_64 Linux VM, 7.8 GB RAM, Python 3.12.3, same pinned dependencies as the Docker image. The load generator runs *on the same 2 vCPUs*. At about 4–6k req/s the client is competing for CPU, so the Phase 3 peak numbers are a lower bound.
- **How the stacks ran:** the sandbox used for the measurements could not pull images from Docker Hub. Both systems were therefore started with the exact container commands: Phase 2 as `uvicorn backend.app.main:app` from the Milestone 2 commit, Phase 3 as `gunicorn -c backend/gunicorn.conf.py` plus `redis-server` with the compose flags. `benchmarks/run_load_suite.sh` repeats the whole suite with `docker compose`.
- **Model benchmark** (`benchmarks/bench_model.py`): the same 2,000 requests (validation houses, 10 form fields) go to every variant, each in a fresh process. Latency is best of 3 runs. Memory is RSS after import and model load.
- **Load test** (`benchmarks/load_test.py`): closed loop, $C = 1 / 8 / 32 / 64$ virtual users, 2 s warm-up then 8 s measured, per configuration. **Cold** traffic means every house is unique (0% cache hits). **Warm** traffic draws from 50 popular houses with Zipf popularity, which is how the UI behaves (pre-filled defaults, small tweaks). Server memory is the RSS of the whole process tree, sampled every 0.5 s.
- **Ablation:** each optimisation is an environment switch on the same code (`MODEL_BACKEND`, `WEB_CONCURRENCY`, `BATCHING_ENABLED`, `CACHE_L1_SIZE`, `REDIS_URL`), so each one is measured on its own.

2 · Week 5: model-level optimisation

Profile of the naive model. One prediction takes about 10 ms, and almost none of that is the trees. Most of the time goes to building a 79-column pandas DataFrame and running the scikit-learn `ColumnTransformer`, which

produces 285 one-hot and scaled features, before crossing into xgboost. The fix is to remove that overhead from the request path.

2.1 Model format optimisation: ONNX export (`backend/scripts/export_onnx.py`)

The whole pipeline (imputers, scaler, one-hot encoder and XGBoost) becomes one ONNX graph run by `onnxruntime`, with graph optimisations at `ORT_ENABLE_ALL` and one intra-op thread per worker. A naive conversion was **silently wrong**, with errors up to 0.9 in log price. Three fixes were needed:

- **Sparse-zero semantics.** The ColumnTransformer emits a sparse matrix, and XGBoost treats absent sparse entries as *missing*, not 0. ONNX is dense, so a `Where(x==0, NaN, x)` node is inserted before the tree ensemble.
- **Scaler precision.** XGBoost's histogram split thresholds coincide with training values, so a float32 rounding difference of 1 ulp flips branches. The scaler therefore runs in float64 with the exact `mean_/scale_` constants.
- **String imputer.** `skl2onnx` only converts string-valued missing markers, so categorical NaN becomes `""`.

The export script asserts parity on all 2,919 train and test rows: $\max |\Delta| = 1.8\text{e-}5$ (log price), about \$3 on a typical house.

2.2 Model distillation (`backend/scripts/distill_student.py`)

The API accepts only 10 fields, and the other 69 columns are *always* the frozen defaults, so the teacher's effective input space at serving time is 10-dimensional. The student is an XGBoost model with 150 trees at depth 4, versus the teacher's 300 trees (4,648 nodes vs 8,204). It was trained on 60,000 synthetic serving requests drawn from the training split (real, jittered and marginally resampled rows) and labelled by the teacher, then exported to ONNX. It takes a 10-wide float32 vector: no DataFrame, no one-hot. If a request carries any other column, it falls back to the full ONNX pipeline, so no input is ever silently ignored.

Accuracy check	Value
Fidelity: student vs teacher RMSE (log), 10k held-out synthetic requests	0.0150
Fidelity on real validation houses	0.0125
Validation RMSE (log), teacher, serving protocol (10 fields + defaults)	0.2127 (\$48,949)
Validation RMSE (log), student	0.2135 (\$49,209)
Reference: teacher with all 79 real columns (not reachable through the API)	0.1327

The student is 0.4% worse than the teacher on the metric the service actually delivers. The gap between 0.133 and 0.213 comes from the product decision in Phase 2 to ask users for only 10 fields, not from Phase 3.

2.3 Why not quantization or TensorRT

FP16/INT8 quantization targets matrix multiplications in neural networks. A tree ensemble is a set of float comparisons (`ONNX TreeEnsembleRegressor`), which `onnxruntime`'s quantizer does not support. Lowering threshold precision would also reintroduce the branch-flip error from §2.1. TensorRT needs an NVIDIA GPU, and at about 11 μs per prediction a GPU transfer would cost more than it saves.

2.4 Benchmark results

Variant	Artefact KB	Load s	RSS MB	p50 µs	p95 µs	p99 µs	Speed- up	Batch-256 rows/s	Val RMSE	Max Δ vs naive
Naive (Phase 2): sklearn pickle + pandas	494	1.19	176	10,456	14,909	17,270	1×	13,340	0.2127	0.0e+00
ONNX full pipeline	290	0.12	70	313	570	692	33×	82,585	0.2127	1.2e-05
Distilled student, native xgboost	296	1.40	167	88	196	305	118×	508,831	0.2135	4.3e-02
Distilled student + ONNX (deployed)	143	0.08	67	11	15	32	917×	233,812	0.2135	4.3e-02

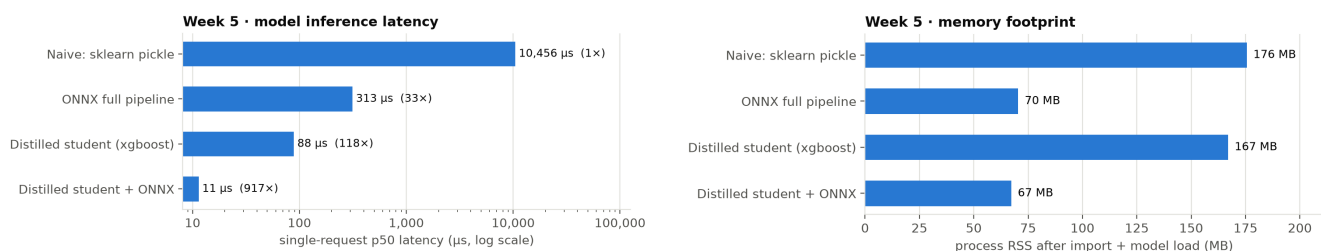


Figure 1. Left: single-request latency (log scale). Right: process memory after import and model load. The ONNX variants no longer import pandas, scikit-learn or xgboost, which cuts about 105 MB per process.

3 · Week 6: system and infrastructure optimisation

3.1 Bottlenecks in Phase 2

- **One process, CPU-bound work under the GIL.** Phase 2 runs one uvicorn process and a synchronous `/predict` in a thread pool. Throughput is capped at about 75 req/s whatever the concurrency, so extra clients only queue: p50 grows from 12 ms to 975 ms at 64 clients.
- **Fixed per-call overhead.** Scoring 256 rows costs little more than scoring 1 (Week 5 table, batch column), so serving one house per model call wastes most of the work.
- **Repeated inputs.** The UI pre-fills every field, so many requests are identical and get recomputed each time.

3.2 Techniques implemented

- **Concurrency and workers** (`backend/gunicorn.conf.py`, `Dockerfile` `CMD`): Gunicorn manages `WEB_CONCURRENCY` Uvicorn workers, one per core by default. Each worker loads the 143 KB model after fork, because ONNX Runtime sessions are not fork-safe. Each worker also has its own async event loop and uses exactly one ONNX thread to avoid oversubscription. `/predict` is now `async`.
- **Dynamic batching** (`backend/app/batching.py`): concurrent requests go into an asyncio queue. A collector groups up to 64 rows and scores them in one `predict_many` call on a dedicated model thread, so the event loop keeps accepting requests. Under load in the full configuration, the average batch was 2.9 rows and the largest was 44. There is also `POST /predict/batch` for clients that batch on their side (up to 256 houses).
- **Caching** (`backend/app/cache.py`, Redis in `docker-compose.yml`): an exact-match cache keyed on `sha256` of the canonical JSON plus the model backend and version. **L1** is a per-worker in-process LRU of 10k entries. **L2** is Redis 7 (LRU, 128 MB, no persistence), shared by all workers and surviving restarts. Writes to Redis run in the background. A circuit breaker bypasses a dead Redis, so the service never depends on it; this is covered by a test.

3.3 Results: cold traffic (raw capacity, 0% cache hits)

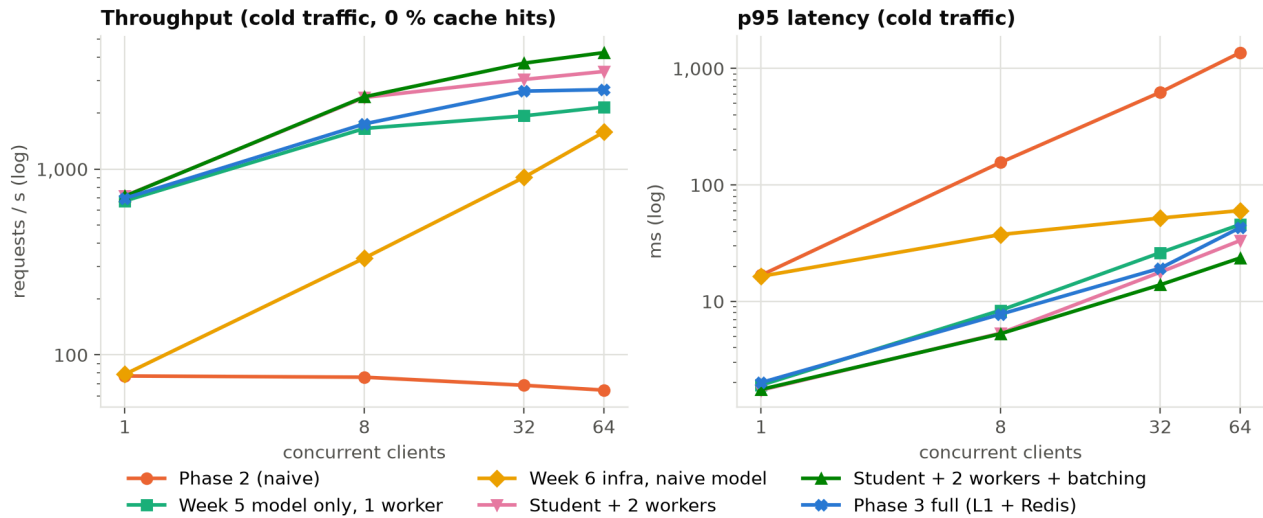


Figure 2. Throughput and p95 latency against concurrent clients, one line per configuration (log axes).

Configuration	req/s @1	p50 ms @1	req/s @64	p50 ms @64	p95 ms @64	p99 ms @64	RSS MB
Phase 2: 1 uvicorn worker, sklearn pickle	77	12.48	65	975.5	1,368.8	1,509.0	216
W5 only: student ONNX, 1 worker	674	1.41	2,158	28.2	45.7	60.2	130
W6 only: sklearn pickle, 2 workers + batching	79	12.24	1,583	37.3	60.3	119.1	433
Student ONNX + 2 Gunicorn workers	718	1.34	3,354	17.5	33.3	50.5	228
+ dynamic batching (greedy)	715	1.36	4,240	14.3	23.6	41.1	225
(batching with 2 ms wait window)	260	3.75	3,834	15.6	26.9	47.6	225
+ L1 LRU + Redis cache (Phase 3 full)	697	1.34	2,680	21.8	43.0	74.5	249

3.4 Results: warm traffic (caching)

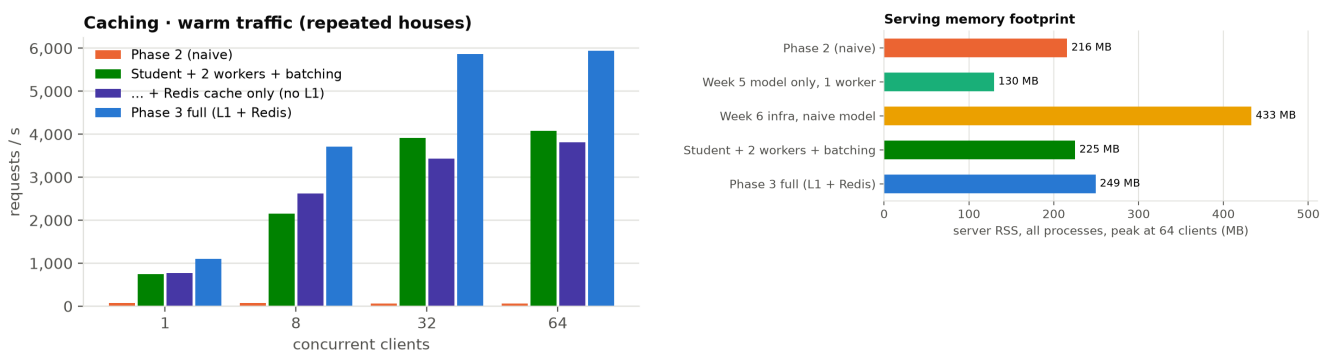


Figure 3. Left: throughput with repeated houses. Right: peak server memory at 64 clients, all processes including the Gunicorn master.

Warm traffic	req/s @8	p95 ms @8	req/s @64	p95 ms @64	p99 ms @64	hit rate
Phase 2 (no cache)	74	169.6	68	1,273.3	1,385.0	0%
Phase 3 without cache (recompute)	2,155	6.4	4,080	24.6	44.2	0%
Redis only (L2)	2,621	5.1	3,814	28.2	50.2	100%
L1 in-process LRU + Redis (deployed)	3,711	4.1	5,947	18.0	22.2	100%

3.5 What the ablation shows

- **Week 5 alone** (1 worker, student model) raises capacity from 65 to 2,158 req/s. Once the model is cheap, the remaining cost is HTTP, JSON and validation.
- **Week 6 alone** (naive sklearn model with workers and batching) raises capacity from 65 to 1,583 req/s (24×). Batching spreads the pandas and ColumnTransformer overhead across many rows, so each technique helps without the other.
- **Workers:** 2 processes give 1.55× the throughput of 1 worker on 2 vCPUs, below the ideal 2× because the load generator uses the same cores.
- **Batching:** +26% throughput and 19% lower p99 at 64 clients. A first version waited up to 2 ms for a batch to fill, which cut single-client throughput from 718 to 260 req/s. The deployed *greedy* mode (`BATCH_MAX_WAIT_MS=0`) never waits. It batches whatever queued up while the model thread was busy, so batches form only under load and there is no penalty at low load (715 req/s at 1 client).
- **Caching:** once the model takes about 11 μs, a Redis round trip costs more than inference. Redis alone was *slower* than recomputing at 64 clients (3,814 vs 4,080 req/s). Adding the in-process L1 fixed this: 5,947 req/s, +46% over no cache, with p99 at 64 clients down from 44 to 22 ms. Redis stays as the shared L2, so a new or restarted worker starts warm and Phase 2-style expensive backends still benefit.

Trade-offs and issues found

- On traffic that is entirely unique, the cache is pure overhead: one Redis GET on every miss, with Redis sharing the same 2 vCPUs. Cold throughput at 64 clients is 2,680 req/s with the cache and 4,240 without it. Both tiers can be turned off (`CACHE_L1_SIZE=0`, `REDIS_URL=`) for a workload like that. On a real deployment Redis would run on its own cores.
- Serving memory: each worker needs about 130 MB, down from 216 MB for Phase 2's single process. Running 2 workers plus the master uses 249 MB in total, which buys the 41× throughput.
- Unicorn's `max_requests` worker recycling dropped 64 open keep-alive connections mid-test in an early run. Recycling is now opt-in (`MAX_REQUESTS`, default 0).

4 · Architecture of the optimised system

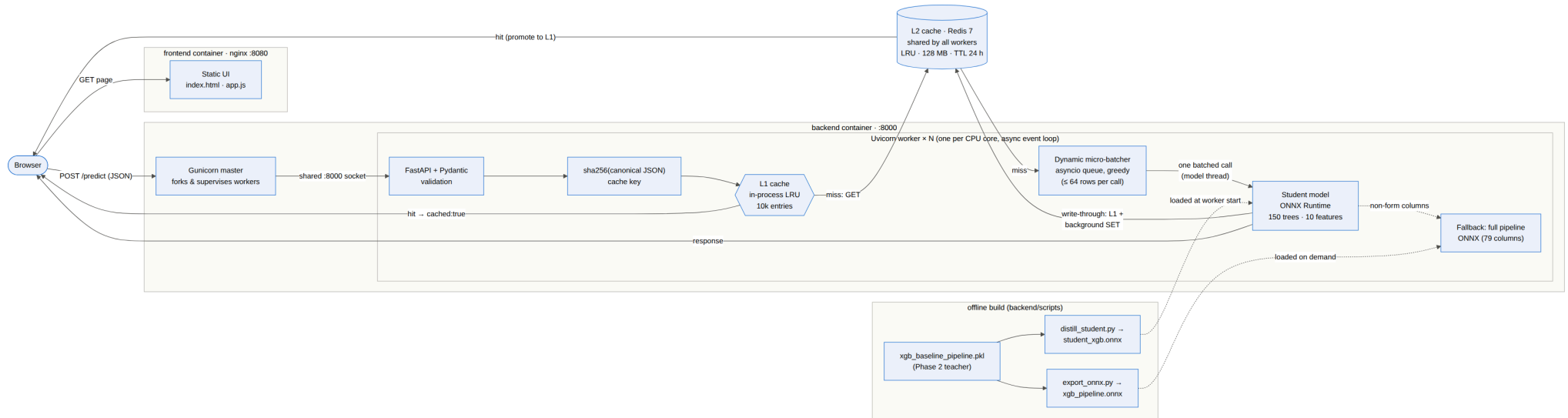


Figure 4. Request flow in Phase 3 (Mermaid source: `docs/architecture.mmd`). Solid arrows are the request path. Dotted arrows are artefacts built offline and loaded at start-up.

Request path. The browser loads the static UI from nginx, then POSTs to the backend. Gunicorn's workers accept on the shared `:8000` socket. FastAPI and Pydantic validate the body and it is hashed into a cache key. The request then checks L1 (in-process LRU), then L2 (Redis, promoted into L1 on a hit), and on a miss joins the micro-batcher queue. One batched call runs the distilled ONNX student on the model thread. The result goes back to the client and is written through to L1 and, in the background, to Redis. Requests with non-form columns go to the full-pipeline ONNX model.

Changed files and reproduction

```
backend/app/model.py      pluggable backends: sklearn | onnx | student (+ fallback)
backend/app/batching.py  dynamic micro-batcher      backend/app/cache.py    L1 LRU + Redis L2
backend/app/main.py      async /predict, /predict/batch, /stats
backend/gunicorn.conf.py multi-worker serving      docker-compose.yml     + redis service
backend/scripts/export_onnx.py, distill_student.py  models/*.onnx, *.json  new artefacts
benchmarks/bench_model.py, load_test.py, run_load_suite.sh, make_report.py, results/
backend/tests/test_phase3.py (17 new tests, 68 total, all passing)
python backend/scripts/export_onnx.py && python backend/scripts/distill_student.py
python benchmarks/bench_model.py      # Week 5 benchmark
./deploy.sh                           # stack: frontend + backend (Gunicorn) + Redis
./benchmarks/run_load_suite.sh         # Week 6 load tests with docker compose
python benchmarks/make_report.py       # charts + summary.md
```